

Chapter 39

Evaluation, Tuning, Interpretation & Honesty

Build a confusion matrix and compute precision, recall, F1, and specificity from it by hand · draw and read ROC and precision-recall curves, and know which one to trust on imbalanced data · judge a regression model with MAE, RMSE, MAPE, and R^2 , and know what each hides

Approved chapter · v1

Contents

Chapter 39. Evaluation, Tuning, Interpretation & Honesty

Why this matters

In plain English

39.1 The confusion matrix

39.2 Curves: ROC and precision-recall

39.3 Regression metrics

39.4 Calibration: are the probabilities honest?

39.5 Choosing the threshold by business cost

39.6 Imbalanced data

39.7 Interpretation: SHAP and partial dependence

39.8 Fairness checks

39.9 Model cards

Common mistakes and how to spot them

In the real world: the rep who scored lower

Tools

The project: cost-based evaluation and explanation of the lead-scoring model

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 39. Evaluation, Tuning, Interpretation & Honesty

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: build a confusion matrix and compute precision, recall, F1, and specificity from it by hand · draw and read ROC and precision–recall curves, and know which one to trust on imbalanced data · judge a regression model with MAE, RMSE, MAPE, and R^2 , and know what each hides · check whether predicted probabilities are honest (calibration) and fix them when they aren't · choose a decision threshold from business costs and team capacity, not from a default of 0.5 · handle imbalanced classes with class weights and resampling, and know what they do and don't change · explain a model globally and one prediction at a time with SHAP and partial dependence · check whether a model treats groups of people fairly · write a model card that says what a model is for and where it fails.

Before you start: Chapter 35 (log loss and the base-rate benchmark), Chapter 36 (the lead-scoring pipeline and validation set), Chapter 37 (the churn models). This chapter evaluates the models you already built.

Time needed: 10–14 hours over two weeks.

Tools: Python 3 with scikit-learn, plus `shap` and `imbalanced-learn` (both free).

Practice data: the lead-scoring model and validation set from Chapter 36 (2,225 leads, 146 won), and the churn and revenue models from Chapter 37. Every number in this chapter was calculated, and every output shown is real.

Why this matters

A model's score is where most machine learning conversations start and stop: "It's 82% accurate." "AUC is 0.82." This chapter is about what those numbers hide and what a business actually needs to know:

- Chapter 36's lead model has an AUC of 0.823. **How many leads should the sales team call?** AUC can't say.
- The model predicts a 20% chance of winning a lead. **Does it win one in five of those?** That's calibration, and one of Chapter 37's models fails it badly.
- **Why did this lead score 65%?** A rep will ask, and "the model said so" is not an answer.
- **Does the model score inside-sales leads fairly?** Someone will ask that too, and the answer here is not simple.

- Predicting "no churn" for everyone is 90% accurate. **Accuracy is the wrong number** on any imbalanced problem, and most business problems are imbalanced.

The chapter's title includes *honesty* because that's the skill being taught: reporting what a model can and can't do, in units a manager understands, before anyone depends on it.

In plain English

Think of a smoke detector.

It can fail two ways: it goes off when there's no fire (a **false positive**: annoying, you open a window), or it stays silent during a fire (a **false negative**: catastrophic). Counting the four outcomes, alarm-fire, alarm-no fire, silent-fire, silent-no fire, is a **confusion matrix**.

How sensitive should it be? Turn it up and you catch every fire but it shrieks at toast. Turn it down and it's quiet but misses a smouldering sofa. That dial is the **threshold**, and the right setting depends on how much a false alarm costs against a missed fire. That's **threshold selection by cost**. A model that reports "70% chance of fire" should be right about 70% of the time when it says that; if it says 70% and there's a fire only 10% of the time, it's **poorly calibrated**. When the alarm goes off, you'd like to know why: smoke, heat, or a low battery. That's **interpretation**. And if the detector works in the kitchen but never in the bedroom, that's a **fairness** problem someone should have tested for.

39.1 The confusion matrix

Four numbers, then everything else

Start from Chapter 36's lead model on its validation set: 2,225 leads from the first half of 2025, of which 146 were won. The model gives each lead a probability; a **threshold** turns it into a yes-or-no decision, and comparing decisions with outcomes gives four counts:

	Actually won	Actually lost
Predicted won	True positive (TP)	False positive (FP)
Predicted lost	False negative (FN)	True negative (TN)

```

import sys
import warnings

import numpy as np
import pandas as pd

sys.path.append(".")
from lead_data import LEAD_CATS, LEAD_NUMS, load_leads, make_model

warnings.filterwarnings("ignore", category=UserWarning)

train, valid, test = load_leads()
X_cols = LEAD_CATS + LEAD_NUMS
model = make_model(LEAD_CATS, LEAD_NUMS).fit(train[X_cols], train["won"])
p_valid = model.predict_proba(valid[X_cols])[:, 1]
y_valid = valid["won"].to_numpy()
print(f"{len(valid):,} validation leads, {y_valid.sum()} won ({y_valid.mean():.2%}")
print(
    f"predicted probabilities: min {p_valid.min():.3f}, median {np.median(p_valid):.3f}, "
    f"max {p_valid.max():.3f}"
)

```

```

2,225 validation leads, 146 won (6.56%)
predicted probabilities: min 0.001, median 0.037, max 0.646

```

```

from sklearn.metrics import confusion_matrix

def matrix_at(threshold):
    predicted = (p_valid >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_valid, predicted).ravel()
    return tp, fp, fn, tn

for threshold in [0.5, 0.2, 0.1]:
    tp, fp, fn, tn = matrix_at(threshold)
    print(
        f"threshold {threshold}: TP {tp:>4}  FP {fp:>5}  FN {fn:>4}  TN {tn:>5}    "
        f"accuracy {(tp + tn) / len(y_valid):.3f}"
    )

```

```

threshold 0.5: TP    4  FP    3  FN 142  TN 2076  accuracy 0.935
threshold 0.2: TP   55  FP   122  FN  91  TN 1957  accuracy 0.904
threshold 0.1: TP  100  FP   386  FN  46  TN 1693  accuracy 0.806

```

Reading it. At the default threshold of 0.5, the model predicts "won" for 7 of 2,225 leads. It's right about 4, and misses 142 of the 146 wins. Its accuracy is 93.5%, which is *worse* than predicting "lost" for everyone (93.4%, since $2,079 \div 2,225 = 0.934$). Accuracy tells you nothing here. The problem isn't the model: only 6.6% of leads are won, so a well-calibrated model rarely says a lead is *more likely than not* to be won. The threshold has to move.

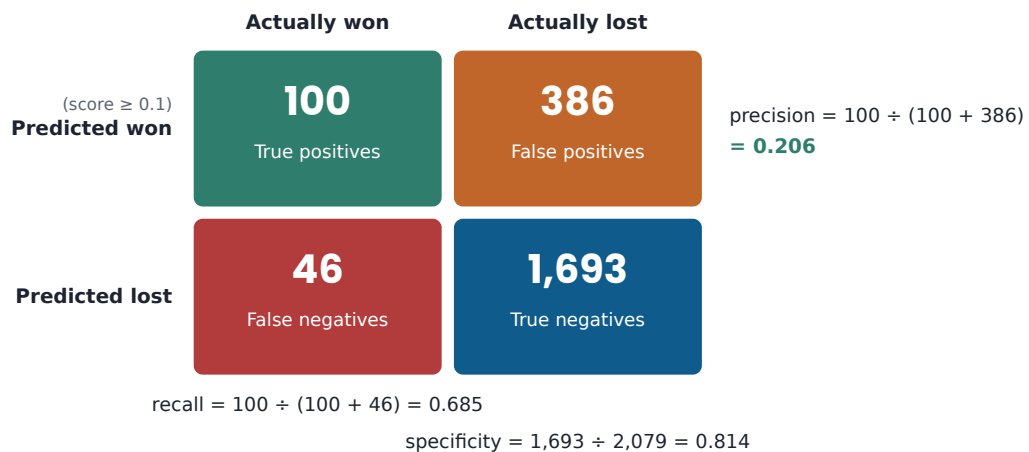


Figure 39.1 – The confusion matrix at a threshold of 0.1. Precision reads across the top row; recall reads down the first column.

Precision, recall, F1, and the rest

From the four counts at a threshold of 0.1 (TP 100, FP 386, FN 46, TN 1,693):

precision = $TP \div (TP + FP) = 100 \div 486 = 0.206$: of the leads the model flags, one in five is won

recall (sensitivity, true positive rate) = $TP \div (TP + FN) = 100 \div 146 = 0.685$: of the leads that are won, the model flagged two thirds

F1 = $2 \times \text{precision} \times \text{recall} \div (\text{precision} + \text{recall}) = 2 \times 0.206 \times 0.685 \div 0.891 = 0.317$ (0.316 from the unrounded precision and recall): the harmonic mean, which punishes lopsided pairs

specificity (true negative rate) = $TN \div (TN + FP) = 1,693 \div 2,079 = 0.814$; its complement, the **false positive rate**, is 0.186

```
from sklearn.metrics import f1_score, precision_score, recall_score

tp, fp, fn, tn = matrix_at(0.1)
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1 = 2 * precision * recall / (precision + recall)
specificity = tn / (tn + fp)
print(
    f"at threshold 0.1: precision {precision:.3f}    recall {recall:.3f}    "
    f"F1 {f1:.3f}    specificity {specificity:.3f}"
)
predicted = (p_valid >= 0.1).astype(int)
print(
    f"scikit-learn:    precision {precision_score(y_valid, predicted):.3f}    "
    f"recall {recall_score(y_valid, predicted):.3f}    F1 {f1_score(y_valid, predicted):.3f}"
)
```

```
at threshold 0.1: precision 0.206    recall 0.685    F1 0.316    specificity 0.814
scikit-learn:    precision 0.206    recall 0.685    F1 0.316
```

Precision and recall pull against each other. Raise the threshold and you flag fewer leads, more of which are won (higher precision) but fewer of all the wins (lower recall). Which matters more depends entirely on the decision: a fraud alert that freezes a card wants precision; a cancer screen wants recall. F1 is a compromise for when you have no cost information; section 39.4 replaces it with something better.

39.2 Curves: ROC and precision–recall

A single threshold gives a single matrix. Sweeping the threshold from 1 down to 0 traces a curve, and the area under it summarizes ranking quality without choosing a threshold.

- The **ROC curve** plots recall (true positive rate) against the false positive rate. **ROC-AUC** is the area under it: the probability that a random won lead scores higher than a random lost one (Chapter 36).
- The **precision–recall (PR) curve** plots precision against recall. **PR-AUC**, usually reported as **average precision**, is the area under it.

```
from sklearn.metrics import (
    average_precision_score,
    precision_recall_curve,
    roc_auc_score,
    roc_curve,
)

fpr, tpr, roc_thresholds = roc_curve(y_valid, p_valid)
prec, rec, pr_thresholds = precision_recall_curve(y_valid, p_valid)
print(
    f"ROC-AUC {roc_auc_score(y_valid, p_valid):.3f}    "
    f"PR-AUC (average precision) {average_precision_score(y_valid, p_valid):.3f}    "
    f"base rate {y_valid.mean():.3f}"
)
print(f"the ROC curve has {len(fpr)} points; the PR curve has {len(prec)}")
random_pr = y_valid.mean()
print(f"a random model would score PR-AUC {random_pr:.3f} and ROC-AUC 0.500")
```

```
ROC-AUC 0.823   PR-AUC (average precision) 0.302   base rate 0.066
the ROC curve has 240 points; the PR curve has 2226
a random model would score PR-AUC 0.066 and ROC-AUC 0.500
```

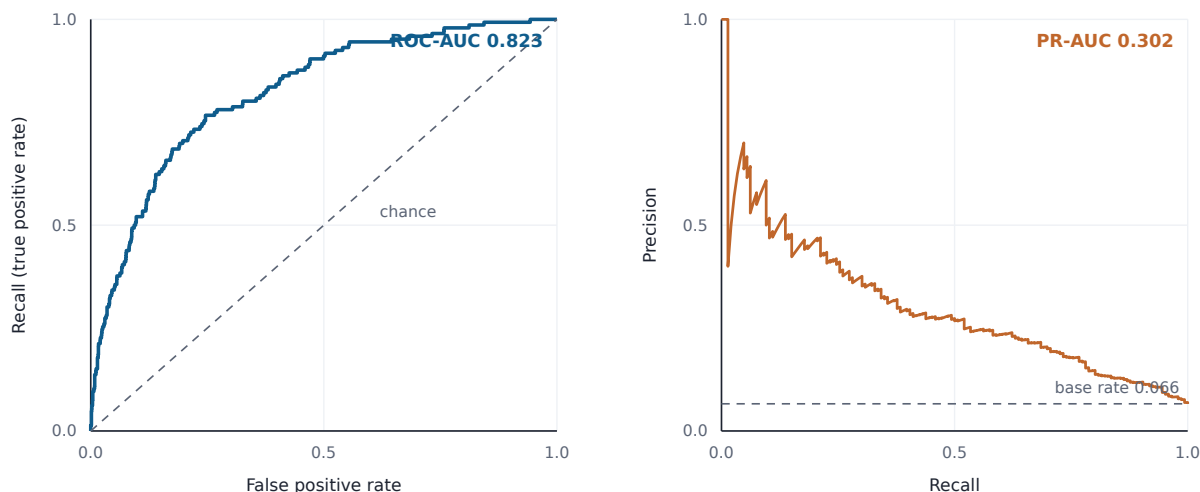


Figure 39.2 – ROC (left) and precision–recall (right) curves for the lead model. The ROC curve looks impressive; the PR curve shows what the sales team will experience: precision falls fast as they call further down the list.

Reading it. ROC-AUC is 0.823 and looks good. PR-AUC is 0.302, which looks poor, and both are correct. The difference is the base rate. A random model scores 0.5 on ROC-AUC regardless of imbalance, but only 0.066 on PR-AUC here (the base rate). The lead model's PR-AUC is 4.6 times a random model's, which is a real achievement, and it still means that a rep working the list will hear "no" most of the time.

Which to report. On imbalanced problems, **PR-AUC is the more honest summary**, because it's built from precision, the number the person acting on the model feels. ROC-AUC is fine for comparing models and for balanced problems. Report both, and say what the base rate is.

The list, not the curve

The most useful evaluation for a call list isn't a curve at all. It's "if we call the top N, how many do we win?"

```
order = np.argsort(-p_valid)
for k in [50, 100, 200, 400]:
    top = order[:k]
    print(
        f"top {k:>3} leads by score: {y_valid[top].sum():>3} won "
        f"({y_valid[top].mean():.1%} of them; "
        f"{y_valid[top].sum() / y_valid.sum():.1%} of all wins)"
    )
```

```
top  50 leads by score:  22 won (44.0% of them; 15.1% of all wins)
top 100 leads by score:  38 won (38.0% of them; 26.0% of all wins)
top 200 leads by score:  59 won (29.5% of them; 40.4% of all wins)
top 400 leads by score:  92 won (23.0% of them; 63.0% of all wins)
```

Reading it. The top 50 leads by score are won 44% of the time, nearly seven times the 6.6% base rate. The top 400 (18% of leads) contain 63% of all the wins. This table is what to show a sales head, because it answers the question they'll ask, and it leads straight into section 39.4.

39.3 Regression metrics

For models that predict a number, the confusion matrix has no equivalent; the errors are amounts. Chapter 37's model predicting each account's 2025 revenue from its 2024 revenue gives a worked example:

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_absolute_percentage_error,
    r2_score,
    root_mean_squared_error,
)
from sklearn.model_selection import train_test_split

accounts = pd.read_csv("../accounts/accounts.csv")
stayed = accounts[accounts["churned_2025"] == 0].copy()
stayed["log_revenue_2024"] = np.log(stayed["revenue_2024"])
stayed["log_revenue_2025"] = np.log(stayed["revenue_2025"])
reg_train, reg_test = train_test_split(stayed, test_size=0.25, random_state=37)
reg = LinearRegression().fit(
    reg_train[["log_revenue_2024"]], reg_train["log_revenue_2025"]
)
pred_rupees = np.exp(reg.predict(reg_test[["log_revenue_2024"]]))
actual = reg_test["revenue_2025"]
print(f"MAE    ₹{mean_absolute_error(actual, pred_rupees):>12,.0f}")
print(f"RMSE    ₹{root_mean_squared_error(actual, pred_rupees):>12,.0f}")
print(f"MAPE     {mean_absolute_percentage_error(actual, pred_rupees):>12.1%}")
print(
    f"R²        {r2_score(actual, pred_rupees):>12.3f}    (R² on log scale "
    f"    {r2_score(reg_test['log_revenue_2025'], np.log(pred_rupees)):.3f})"
)
errors = (actual - pred_rupees).abs().sort_values(ascending=False)
print(
    f"largest single error ₹{errors.iloc[0]:,.0f}; the 10 largest errors are "
    f"    ₹{errors.iloc[:10].sum() / errors.sum():.1%} of the total absolute error"
)
```

```
MAE    ₹      92,933
RMSE    ₹     232,567
MAPE           19.2%
R²           0.927    (R² on log scale 0.955)
largest single error ₹3,697,812; the 10 largest errors are 15.2% of the total absolute error
```

The four common measures:

- **MAE** (mean absolute error): the average size of a miss, in rupees. ₹92,933 here. Simple, and the right number to quote to a manager: "our forecast is typically off by about ₹93,000 per account".
- **RMSE** (root mean squared error): the square root of the average squared miss. ₹232,567, two and a half times the MAE, because squaring makes big misses dominate: the ten largest errors are 15% of the total error, and one account is off by ₹3.7 million. RMSE is the right measure when large errors are disproportionately costly; otherwise it's dragged around by outliers.
- **MAPE** (mean absolute percentage error): the average miss as a percentage of the actual value. 19.2%. Managers like it because it's scale-free. It fails when actual values are near zero (a ₹100 miss on a ₹200 account is 50%), and it punishes over-prediction more than under-prediction, so it's biased when used to choose a model.

- **R²**: the share of variance explained (Chapter 37). 0.927 in rupees, 0.955 on the log scale the model was fitted on. The same model, two R² values: R² depends on what scale you measure it on, and comparing R² between models fitted to different targets is meaningless.

Watch out: the metric must match the loss. A model trained on log revenue minimizes percentage-type errors, so it does well on MAPE and worse on RMSE in rupees, where the largest accounts dominate. If the business cares about total rupees, train on rupees (or weight by size). Decide what error costs the business before you decide what to optimize.

For forecasting over time, Chapter 40 adds **WAPE** (weighted absolute percentage error), which fixes MAPE's problems and is the standard in demand planning.

39.4 Calibration: are the probabilities honest?

A model can rank well and still lie about probabilities. **Calibration** asks: among leads the model gave a 20% chance, were about 20% won? The check is a **reliability curve**: bin leads by predicted probability, and plot the actual win rate in each bin against the average prediction. A perfectly calibrated model lies on the diagonal.

Two summary numbers: **log loss** (Chapter 35) and the **Brier score**, the mean squared difference between predicted probability and outcome (0 is perfect; predicting the base rate for everyone gives about 0.061 here).

```

from sklearn.calibration import calibration_curve
from sklearn.metrics import brier_score_loss, log_loss
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import FunctionTransformer
from sklearn.ensemble import HistGradientBoostingClassifier

to_dense = FunctionTransformer(lambda X: X.toarray() if hasattr(X, "toarray") else X)
nb = make_model(LEAD_CATS, LEAD_NUMS, GaussianNB())
nb.steps.insert(1, ("dense", to_dense))
boost = make_model(
    LEAD_CATS, LEAD_NUMS, HistGradientBoostingClassifier(random_state=39)
)
probs = {
    "logistic regression": p_valid,
    "Naive Bayes": nb.fit(train[X_cols], train["won"]).predict_proba(valid[X_cols])[
        :, 1
    ],
    "gradient boosting": boost.fit(train[X_cols], train["won"]).predict_proba(
        valid[X_cols]
    )[:, 1],
}
curves = {}
for name, p in probs.items():
    frac_pos, mean_pred = calibration_curve(y_valid, p, n_bins=10, strategy="quantile")
    curves[name] = (mean_pred, frac_pos)
    print(
        f"{name:<20} log loss {log_loss(y_valid, p):.4f} "
        f" Brier {brier_score_loss(y_valid, p):.4f} "
        f"mean prediction {p.mean():.3f} vs actual {y_valid.mean():.3f}"
    )
print("\nNaive Bayes, by decile of predicted probability:")
print(
    pd.DataFrame(
        {
            "mean predicted": curves["Naive Bayes"][0],
            "actual win rate": curves["Naive Bayes"][1],
        }
    )
    .round(3)
    .to_string(index=False)
)

```

logistic regression	log loss 0.1948	Brier 0.0531	mean prediction 0.069 vs actual 0.066
Naive Bayes	log loss 0.4830	Brier 0.1243	mean prediction 0.180 vs actual 0.066
gradient boosting	log loss 0.2075	Brier 0.0556	mean prediction 0.065 vs actual 0.066

Naive Bayes, by decile of predicted probability:

mean predicted	actual win rate
0.000	0.009
0.000	0.014
0.000	0.013
0.000	0.027
0.002	0.018
0.010	0.032
0.045	0.050
0.192	0.076
0.610	0.149
0.939	0.269

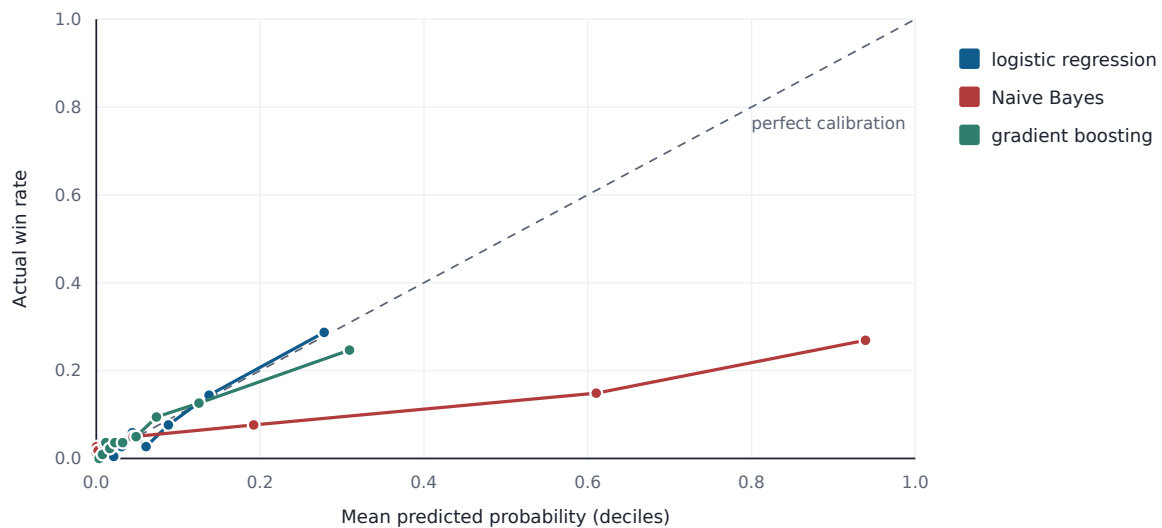


Figure 39.3 – Reliability curves for three models. Logistic regression and gradient boosting are honest; Naive Bayes is confidently wrong in both directions.

Reading it. Logistic regression and gradient boosting are well calibrated: their average prediction (6.9% and 6.5%) matches the actual win rate (6.6%), and their bins sit near the diagonal. Naive Bayes is a different story. Its average prediction is **18%**, nearly three times the real rate, and its top decile predicts a **94%** chance of winning for leads that are won **27%** of the time. It ranks reasonably (ROC-AUC 0.806) and its probabilities are fiction. This is the overconfidence Chapter 37 warned about, and it matters the moment someone uses the number: a rep told "94%" who wins one in four will stop trusting every model.

Fixing calibration

`CalibratedClassifierCV` wraps a model and learns a correction from predicted score to real probability, using cross-validation so it doesn't learn on the same rows twice. **Platt scaling** (`method="sigmoid"`) fits a logistic curve; **isotonic regression** (`method="isotonic"`) fits any rising step function and needs more data.

```
from sklearn.calibration import CalibratedClassifierCV

calibrated_nb = CalibratedClassifierCV(nb, method="isotonic", cv=5).fit(
    train[X_cols], train["won"]
)
p_nb_cal = calibrated_nb.predict_proba(valid[X_cols])[:, 1]
print(
    f"Naive Bayes, isotonic calibration: log loss {log_loss(y_valid, p_nb_cal):.4f}  "
    f"Brier {brier_score_loss(y_valid, p_nb_cal):.4f}  "
    f"ROC-AUC {roc_auc_score(y_valid, p_nb_cal):.3f}"
)
print(
    f"(before calibration: ROC-AUC {roc_auc_score(y_valid, probs['Naive Bayes']):.3f})"
```

```
Naive Bayes, isotonic calibration: log loss 0.2022  Brier 0.0544  ROC-AUC 0.802
(before calibration: ROC-AUC 0.806)
```

Reading it. After isotonic calibration, Naive Bayes's log loss drops from 0.483 to 0.202 and its Brier score from 0.124 to 0.054, close to logistic regression's. Its ranking barely changes (ROC-AUC 0.806 to 0.802), because calibration reshapes the scores without reordering them much. **Calibration fixes the numbers, not the ranking.**

When calibration matters: whenever the probability is used as a number: expected value calculations (the next section), thresholds by cost, combining models, showing a percentage to a person. When only the order matters, such as a call list, it doesn't.

39.5 Choosing the threshold by business cost

The two numbers you need

A threshold is a business decision disguised as a model setting. To set it, you need what a false positive and a true positive are worth. For lead scoring, working a lead properly costs time, and winning one earns margin:

```
WORK_COST = (
    1500 # ₹ to work a lead properly: calls, a quote, samples (about 2 rep-hours)
)
new_accounts = accounts[accounts["tenure_months"] <= 12]
FIRST_YEAR_REVENUE = new_accounts["revenue_2024"].median()
MARGIN = 0.15
WIN_VALUE = FIRST_YEAR_REVENUE * MARGIN
print(
    f"{len(new_accounts)} accounts in their first "
    f"year: median revenue ₹{FIRST_YEAR_REVENUE:,.0f}"
)
print(f"value of a won lead at {MARGIN:.0%} margin: ₹{WIN_VALUE:,.0f}")
print(f"break-even win probability: {WORK_COST / WIN_VALUE:.4f}")
```

```
878 accounts in their first year: median revenue ₹201,700
value of a won lead at 15% margin: ₹30,255
break-even win probability: 0.0496
```

How it works: the cost to work a lead, ₹1,500, covers calls, a quote, and samples, about two hours of a rep's time; change it to your own number. The value of a win is the median first-year revenue of Riverstone's newer accounts (₹201,700, from the accounts data) times a 15% gross margin: ₹30,255. Both are stated assumptions; the point of writing them as named variables is that the sales head can argue with them and re-run the analysis.

Break-even. Working a lead pays off when $p \times ₹30,255 > ₹1,500$, so $p > 1,500 \div 30,255 = \mathbf{0.0496}$. Any lead with a calibrated probability above about 5% is worth working. That number came from costs, not from the model, and it's nowhere near 0.5.

The profit curve

```
def profit_at(threshold, p=p_valid, y=y_valid):
    called = p >= threshold
    return (
        WIN_VALUE * y[called].sum() - WORK_COST * called.sum(),
        called.sum(),
        y[called].sum(),
    )

print("threshold   worked   wins   profit")
for threshold in [0.0, 0.02, 0.03, 0.05, 0.07, 0.10, 0.15, 0.20, 0.50]:
    profit, worked, wins = profit_at(threshold)
    print(f"{threshold:>9.2f}   {worked:>6}   {wins:>4}   ₹{profit:>10,.0f}")

grid = np.linspace(0.0, 0.5, 501)
profits = np.array([profit_at(t)[0] for t in grid])
best = grid[profits.argmax()]
print(f"\nbest threshold on the grid: {best:.3f}   profit ₹{profits.max():,.0f}")
print(f"working every lead: ₹{profit_at(0.0)[0]:,.0f}   working none: ₹0")
```

threshold	worked	wins	profit
0.00	2225	146	₹ 1,079,730
0.02	1476	138	₹ 1,961,190
0.03	1239	135	₹ 2,225,925
0.05	904	121	₹ 2,304,855
0.07	684	114	₹ 2,423,070
0.10	486	100	₹ 2,296,500
0.15	291	76	₹ 1,862,880
0.20	177	55	₹ 1,398,525
0.50	7	4	₹ 110,520

best threshold on the grid: 0.077 profit ₹2,448,060
 working every lead: ₹1,079,730 working none: ₹0

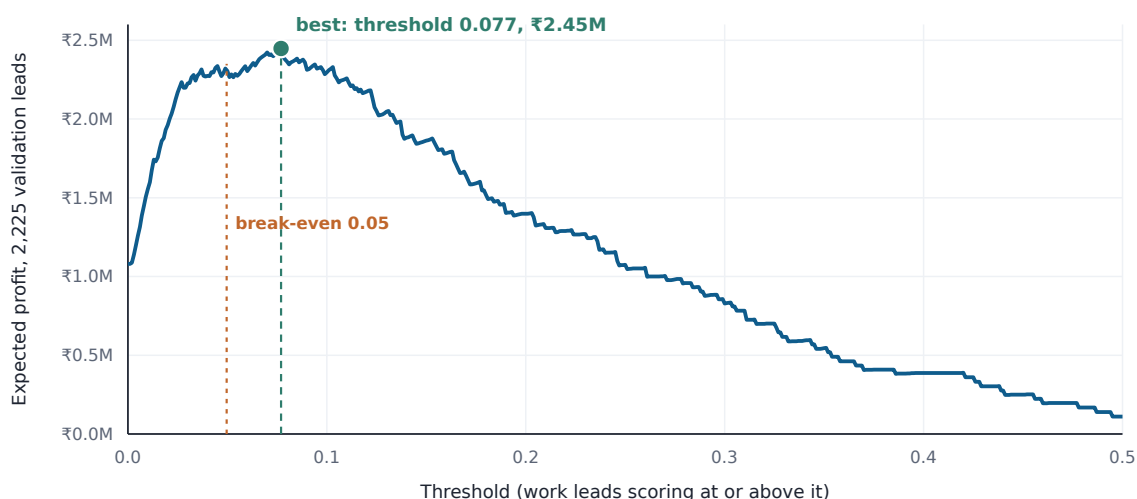


Figure 39.4 — Profit against threshold. Working every lead is profitable but wasteful; the peak is broad, so anything between 0.05 and 0.10 is close to best; above 0.15 the team is leaving money on the table.

Reading it. Working all 2,225 leads makes ₹1.08 million. Working only those above 0.077 makes **₹2.45 million**, from 627 leads instead of 2,225. The peak is broad: thresholds from 0.05 to 0.10 are all within 6% of the best, so precision about the exact number is false precision. Above 0.15 profit falls fast, because the wins the model is now skipping were worth more than the effort saved.

The break-even threshold (0.0496) and the profit-maximizing one (0.077) differ because the model's probabilities near 5% are slightly optimistic, and because the grid is on validation data with only 146 wins. In practice, quote the range.

Capacity, and the old rule

The team can't work 660 leads in six months; it can work about four a day. That's a different constraint, and it sets the threshold from the other direction:

```
DAYS = 126 # working days in the six-month validation period
LEADS_PER_DAY = 4 # what the team can work properly
budget = DAYS * LEADS_PER_DAY
top = np.argsort(-p_valid)[:budget]
threshold_at_budget = p_valid[top].min()
profit, worked, wins = profit_at(threshold_at_budget)
print(
    f"capacity: {budget} leads in the period -> work "
    f"leads scoring above {threshold_at_budget:.3f}"
)
print(f"{worked} worked, {wins} wins, profit ₹{profit:,.0f}")
rule = valid["source"].isin(["Referral", "Trade fair", "Partner"]).to_numpy()
rule_profit = WIN_VALUE * y_valid[rule].sum() - WORK_COST * rule.sum()
print(
    f"the old rule (3 sources): {rule.sum()} worked, "
    f"{y_valid[rule].sum()} wins, profit ₹{rule_profit:,.0f}"
)
best_profit, best_worked, best_wins = profit_at(0.05)
print(
    f"threshold 0.05 with no capacity limit: {best_worked} "
    f"worked, {best_wins} wins, ₹{best_profit:,.0f}"
)
```

```
capacity: 504 leads in the period -> work leads scoring above 0.097
504 worked, 102 wins, profit ₹2,330,010
the old rule (3 sources): 531 worked, 84 wins, profit ₹1,744,920
threshold 0.05 with no capacity limit: 904 worked, 121 wins, ₹2,304,855
```

Reading it. With capacity for 504 leads, the team works everything scoring above 0.097, wins 102, and makes ₹2.33 million. The old rule, "work referrals, trade fairs, and partners", works a similar number of leads (531), wins 84, and makes ₹1.74 million. **The model is worth about ₹590,000 over six months at the same effort**, or roughly ₹1.2 million a year. That's the sentence to put in front of Anita, with the two assumptions next to it.

Watch out: a threshold chosen on validation data is a validation-data threshold. Test it once on the test set, and expect the profit to be a little lower. And monitor it: as the mix of leads changes (Chapter 36 showed the marketplace share rising), the same threshold flags a different number of leads.

39.6 Imbalanced data

With 6.6% positives, the lead problem is **imbalanced**, and there's a large toolkit for that: class weights, undersampling the majority, oversampling the minority, and **SMOTE** (synthetic minority oversampling, which invents new minority rows between existing ones). Here's what each does to the lead model, with `imbalanced-learn`'s pipeline so that resampling happens **inside** training and never touches validation rows:

```
from imblearn.over_sampling import RandomOverSampler, SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.under_sampling import RandomUnderSampler
from sklearn.linear_model import LogisticRegression

def resampled(sampler):
    base = make_model(LEAD_CATS, LEAD_NUMS)
    return ImbPipeline(
        [
            ("prepare", base.named_steps["prepare"]),
            ("sample", sampler),
            ("model", LogisticRegression(max_iter=1000)),
        ]
    )

variants = {
    "plain logistic": make_model(LEAD_CATS, LEAD_NUMS),
    "class_weight='balanced'": make_model(
        LEAD_CATS, LEAD_NUMS, LogisticRegression(max_iter=1000, class_weight="balanced")
    ),
    "random undersampling": resampled(RandomUnderSampler(random_state=39)),
    "random oversampling": resampled(RandomOverSampler(random_state=39)),
    "SMOTE": resampled(SMOTE(random_state=39)),
}

print(f"{'variant':<26} ROC-AUC   PR-AUC   log loss   mean prob   wins in top 200")
for name, pipe in variants.items():
    p = pipe.fit(train[X_cols], train["won"]).predict_proba(valid[X_cols])[:, 1]
    top200 = y_valid[np.argsort(-p)[:200]].sum()
    print(
        f"{'name':<26} {roc_auc_score(y_valid, p):.3f}      "
        f"{'average_precision_score(y_valid, p):.3f}      "
        f"{'log_loss(y_valid, p):.4f}      {p.mean():.3f}      {top200}"
    )
```

variant	ROC-AUC	PR-AUC	log loss	mean prob	wins in top 200
plain logistic	0.823	0.302	0.1948	0.069	59
class_weight='balanced'	0.821	0.300	0.4899	0.353	61
random undersampling	0.814	0.286	0.5067	0.359	58
random oversampling	0.819	0.307	0.4927	0.355	61
SMOTE	0.820	0.297	0.4772	0.339	58

Reading it. Every variant ranks about the same: ROC-AUC 0.814–0.823, PR-AUC 0.286–0.307, and 58–61 wins in the top 200, all within noise. What changes is **the probabilities**: the average prediction jumps from 0.069 to about 0.35, and log loss gets much worse. Resampling teaches the model that wins are five times more common than they are, so it inflates every probability. That helps only if you were going to use a threshold of 0.5 and never think about it. If you choose the threshold by cost (section 39.5), rebalancing gives nothing, and it destroys calibration.

When rebalancing helps: with algorithms that struggle to learn the minority class at all (very deep trees, some neural networks), with extreme imbalance (fraud at 0.1%), and when the minority class is under-represented in *training* relative to production. For logistic regression and boosting on a 7% problem, the honest answer is: **fix the threshold, not the data**. Class weights are the cheapest option if you must, because they need no library and don't change the rows.

39.7 Interpretation: SHAP and partial dependence

Global: which features matter

SHAP values (SHapley Additive exPlanations) assign each feature a share of each prediction, so that base value + all shares = the prediction, in the model's own units (log-odds for a classifier). Averaging their absolute values over many rows gives a global importance that, unlike Chapter 37's impurity importance, is fair to features with few distinct values.

```
import shap

print("shap", shap.__version__)

def dense(matrix):
    return matrix.toarray() if hasattr(matrix, "toarray") else matrix

prepare = model.named_steps["prepare"]
feature_names = prepare.get_feature_names_out()
X_valid_prepared = dense(prepare.transform(valid[X_cols]))
explainer = shap.LinearExplainer(
    model.named_steps["model"], dense(prepare.transform(train[X_cols]))
)
shap_values = explainer.shap_values(X_valid_prepared)
importance = pd.Series(np.abs(shap_values).mean(axis=0), index=feature_names)
print("mean |SHAP| (log-odds units), top 8:")
print(importance.sort_values(ascending=False).head(8).round(3).to_string())
```

```
shap 0.52.0
mean |SHAP| (log-odds units), top 8:
cat__source_Marketplace    0.572
cat__company_size_1-10     0.317
cat__source_Website        0.270
cat__segment_Retail        0.265
num__log_quantity          0.243
num__free_email            0.241
num__activities_24h        0.235
num__text_strong           0.226
```

Reading it. For the lead model, the marketplace source matters most, then very small companies, website leads, and retail segment. The engineered features from Chapter 36 (quantity, free email, early activity, strong wording) follow. Nothing surprising, which is itself reassuring: a model whose top features nobody can explain is a model to distrust.

Local: why this lead scored 65%

```
row = np.argsort(-p_valid)[0] # the highest-scoring validation lead
contributions = pd.Series(shap_values[row], index=feature_names)
print(
    f"lead {valid.iloc[row]['lead_id']}: source {valid.iloc[row]['source']}, "
    f"segment {valid.iloc[row]['segment']}, predicted "
    f"{p_valid[row]:.1%}, actually won: {bool(y_valid[row])}"
)
print(
    f"base value (log-odds of an average lead) {explainer.expected_value:.3f} "
    f"= probability {1 / (1 + np.exp(-explainer.expected_value)):.3f}"
)
shown = contributions[contributions.abs() > 0.15].sort_values()
print(shown.round(3).to_string())
total = explainer.expected_value + contributions.sum()
print(
    f"base + all contributions = {total:.3f} -> probability {1 / (1 + np.exp(-total)):.3f}"
)
```

```
lead 108196: source Referral, segment Wholesale, predicted 64.6%, actually won: True
base value (log-odds of an average lead) -3.293 = probability 0.036
cat__product_interest_Industrial    -0.213
num__website_visits                  -0.197
num__responded_24h                   0.153
cat__source_Website                   0.170
cat__company_size_1-10                0.189
num__activities_24h                   0.201
cat__segment_Retail                   0.223
num__free_email                       0.237
num__log_quantity                     0.249
cat__company_size_200+                0.339
cat__source_Marketplace                0.467
cat__source_Referral                  0.647
num__text_strong                       0.693
base + all contributions = 0.603 -> probability 0.646
```

Reading it. An average lead sits at log-odds -3.293 , a 3.6% chance. This lead's largest pushes are strong wording in its enquiry ($+0.69$), coming through a referral ($+0.65$), not being a marketplace lead ($+0.47$; the one-hot column is 0, and the contribution is relative to the average lead, most of which are marketplace), and being a large company ($+0.34$). Two features pull it down slightly. Adding every contribution to the base gives 0.603, which is a probability of 64.6%, exactly what the model predicted. ✓ That's the sentence for the rep: "Referral, big company, and they asked for a bulk quote."

For a **linear model** these explanations are exact and cheap (`LinearExplainer`). For **tree models**, `TreeExplainer` is exact and fast. For anything else, `KernelExplainer` approximates and is slow; use it on a sample.

Partial dependence: what a feature does on average

```
sys.path.append("../ch37")
CATS = ["segment", "city_tier", "company_size", "rep_id"]
NUMS = [
    "tenure_months",
    "orders_2024",
    "units_2024",
    "revenue_2024",
    "avg_discount_pct",
    "late_payment_days",
    "complaints_2024",
    "categories_bought",
    "days_since_last_order",
    "website_logins_2024",
    "catalog_downloads_2024",
]
accounts["rep_id"] = accounts["rep_id"].astype(str)
acc_train, acc_test = train_test_split(
    accounts, test_size=0.2, random_state=37, stratify=accounts["churned_2025"]
)
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

churn_model = Pipeline(
    [
        # no scaling: trees don't need it, and raw units read better
        (
            "prepare",
            ColumnTransformer(
                [
                    (
                        "cat",
                        OneHotEncoder(handle_unknown="ignore", sparse_output=False),
                        CATS,
                    ),
                    ("num", SimpleImputer(strategy="median", add_indicator=True), NUMS),
                ]
            ),
        ),
        (
            "model",
            HistGradientBoostingClassifier(
                learning_rate=0.05,
                max_depth=3,
                min_samples_leaf=40,
                max_iter=300,
                random_state=37,
            ),
        ),
    ]
)
churn_model.fit(acc_train[CATS + NUMS], acc_train["churned_2025"])
prep_c = churn_model.named_steps["prepare"]
names_c = prep_c.get_feature_names_out()
Xc = prep_c.transform(acc_test[CATS + NUMS])
tree_explainer = shap.TreeExplainer(churn_model.named_steps["model"])
shap_c = tree_explainer.shap_values(Xc)
print("churn model, mean |SHAP|, top 6:")
print(
    pd.Series(np.abs(shap_c).mean(axis=0), index=names_c)
    .sort_values(ascending=False)
    .head(6)
    .round(3)
)
```

```
.to_string()
)
```

```
churn model, mean |SHAP|, top 6:
num__days_since_last_order    0.628
num__categories_bought         0.483
num__late_payment_days         0.289
num__tenure_months              0.282
num__complaints_2024           0.173
num__units_2024                0.151
```

```
from sklearn.inspection import partial_dependence

idx = list(names_c).index("num__days_since_last_order")
pd_result = partial_dependence(
    churn_model.named_steps["model"],
    Xc,
    features=[idx],
    grid_resolution=12,
    kind="average",
    method="brute",
    response_method="predict_proba",
)
print("days since last order -> average predicted churn probability")
for days, value in zip(pd_result["grid_values"][0], pd_result["average"][0]):
    print(f"{days:>6.0f} days    {value:.3f}")
```

```
days since last order -> average predicted churn probability
  2 days    0.065
 24 days    0.047
 45 days    0.039
 67 days    0.036
 88 days    0.066
110 days    0.152
132 days    0.141
153 days    0.142
175 days    0.176
196 days    0.385
218 days    0.385
240 days    0.349
```

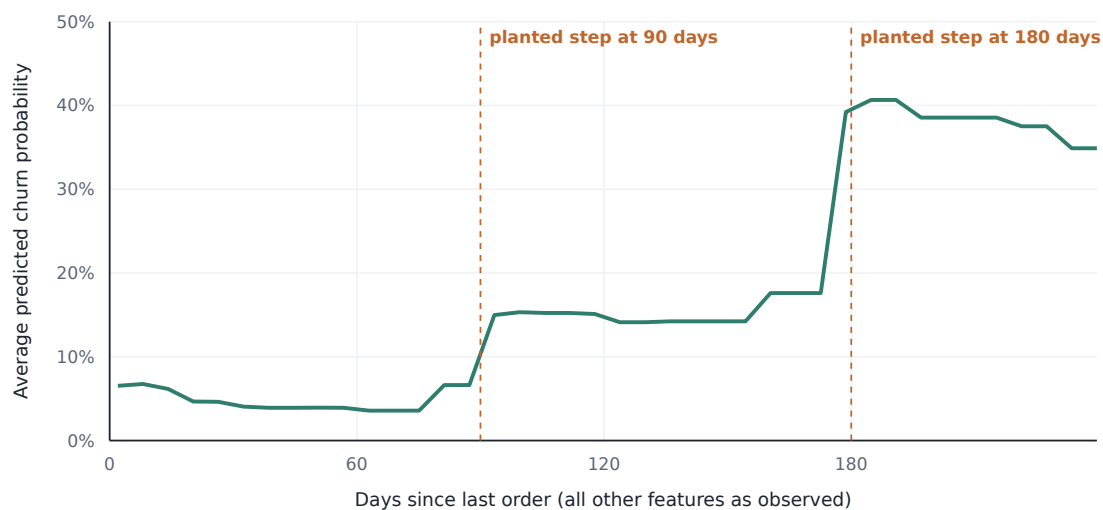


Figure 39.5 — Partial dependence of predicted churn on days since last order, from the gradient-boosting churn model. The two steps at about 90 and 180 days are the thresholds Chapter 37 built into the data, recovered by the model without being told.

Reading it. On the churn model, SHAP ranks days since last order first and product breadth second, as the odds ratios in Chapter 37 did. **Partial dependence** goes further: it fixes one feature at a series of values, averages the model's predictions over all accounts, and shows the feature's effect. Churn risk is flat at about 4–6% for accounts that ordered within 90 days, steps to about 15% between 100 and 180 days, and steps again to about 38% beyond 190 days. Those two steps are exactly what the generator planted. A partial dependence plot is how you check whether a black-box model has learned something sensible or something strange.

Watch out: interpretation is not causation. SHAP explains the *model*, not the customer. "Late payments push churn risk up" means the model uses late payments as a signal, not that reducing late payments reduces churn (Chapter 31). And partial dependence averages over unrealistic combinations (a wholesaler with a hospitality discount); `IndividualConditionalExpectation` plots and SHAP dependence plots show the spread.

39.8 Fairness checks

A model that scores one group of people systematically differently deserves a look, whether the group is defined by protected characteristics, by geography, or, as here, by who handles the lead.

Chapter 36 noted that leads assigned to the inside sales desk convert less often. Compare the two groups at the working threshold of 0.05:

```

valid_scored = valid.assign(p=p_valid, called=(p_valid >= 0.05))
groups = valid_scored.groupby("inside_desk")
report = pd.DataFrame(
    {
        "leads": groups.size(),
        "win rate": groups["won"].mean(),
        "mean score": groups["p"].mean(),
        "share called": groups["called"].mean(),
        "recall": groups.apply(lambda g: g.loc[g["won"] == 1, "called"].mean()),
        "precision": groups.apply(lambda g: g.loc[g["called"], "won"].mean()),
    }
)
report.index = ["sales reps", "inside sales desk"]
print(report.round(3).to_string())

```

	leads	win rate	mean score	share called	recall	precision
sales reps	1289	0.086	0.094	0.552	0.910	0.142
inside sales desk	936	0.037	0.035	0.205	0.571	0.104

Reading it. The model flags 55% of the reps' leads for follow-up and only 21% of the inside desk's. Recall differs sharply: it catches 91% of the reps' eventual wins but only 57% of the desk's. Precision is closer (14% against 10%).

Is that unfair? Three views, and they conflict:

- **The base rates differ.** Inside-desk leads are won 3.7% of the time, reps' leads 8.6%. A calibrated model *must* give lower scores to a group with a lower win rate; if it didn't, it would be miscalibrated for one of them. On this view the model is fair.
- **Equal opportunity** asks whether wins are equally likely to be *caught* in both groups. They aren't: 91% against 57%. Winnable inside-desk leads are being missed at twice the rate.
- **The feedback loop** (Chapter 36, exercise 15): if the desk's leads are flagged less, worked less, and therefore won less, the next model learns an even lower rate for them. The gap can be partly the model's own past doing.

There's no formula that settles this; it's a decision about what Riverstone owes each lead. Reasonable options include a separate threshold for the desk's leads, a floor on how many desk leads are worked, or an experiment (Chapter 30) to learn whether the desk's leads convert better when worked properly. What isn't reasonable is not checking.

```

by_segment = valid_scored.groupby("segment").apply(
    lambda g: pd.Series(
        {
            "leads": len(g),
            "win rate": g["won"].mean(),
            "share called": g["called"].mean(),
            "recall": g.loc[g["won"] == 1, "called"].mean(),
            "false positive rate": g.loc[g["won"] == 0, "called"].mean(),
        }
    )
)
print(by_segment.round(3).to_string())

```

segment	leads	win rate	share called	recall	false positive rate
Hospitality	728.0	0.049	0.419	0.861	0.396
Retail	941.0	0.051	0.317	0.812	0.290
Wholesale	448.0	0.121	0.571	0.815	0.538

By segment, the picture is what you'd expect from a calibrated model with different base rates: wholesale leads (12% win rate) are flagged more and have a higher false positive rate. Recall is similar across segments (81–86%). Report a table like this for every grouping that could matter, including ones you'd rather not look at.

Fairness vocabulary. *Demographic parity*: the same share flagged in each group. *Equal opportunity*: the same recall in each group. *Predictive parity*: the same precision in each group. *Calibration within groups*: the probabilities are honest for each group separately. When base rates differ, these cannot all hold at once; that's a mathematical fact, and choosing between them is a policy decision.

39.9 Model cards

A **model card** is a one-to-two-page document that travels with a model: what it's for, what it was trained on, how it performs, where it fails, and who to call. Here is the lead-scoring model's, condensed:

Section	Lead-scoring model, v1
Purpose	Rank incoming enquiries so reps work the most promising first. Not for pricing, not for deciding which leads to ignore permanently.
Owner	Meera Iyer (analytics); business owner Anita Rao (sales)
Training data	Riverstone CRM, enquiries created Jan 2023–Dec 2024 (7,291 leads, 7.9% won); features as of 24 hours after arrival; duplicates removed; outcome window 90 days
Evaluation	Validation Jan–Jun 2025 (2,225 leads): ROC-AUC 0.823, PR-AUC 0.302, log loss 0.195, well calibrated. Test Jul–Sep 2025 (1,185 leads): ROC-AUC 0.838. Top 50 leads by score: 44% won
Decision rule	Work leads scoring ≥ 0.05 (break-even at ₹1,500 per lead, ₹30,255 per win), capped by capacity of 4 leads per rep-day. Threshold reviewed quarterly
Known limitations	Lower recall on inside-sales-desk leads (57% vs 91%); trained before the 2025 marketplace change, which lowered marketplace conversion; assumes the outcome window and follow-up process stay the same
Fairness	Checked by segment, city, source, and owner (section 39.8); inside-desk gap flagged for policy decision
Monitoring	Weekly: share of leads flagged, predicted vs actual win rate for closed leads; retrain if the calibration drift exceeds 2 points (Chapter 52)
Do not use for	Individual performance reviews of reps; any decision without a human in the loop

The card is short on purpose. Its value is that the "known limitations" and "do not use for" rows exist at all; most model failures in business come from a model being used for something it was never evaluated for.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Reporting accuracy on an imbalanced problem	"93.5% accurate" for a model that flags 7 leads	Report precision, recall, PR-AUC, and the base rate
Leaving the threshold at 0.5	Almost nothing flagged	Choose the threshold from costs or capacity
Reporting only ROC-AUC	Looks great, team disappointed	Add PR-AUC and the top-N table
Comparing R^2 across different targets	"Log model beats rupee model" from R^2 alone	Compare on one scale, in the business's units
Quoting RMSE without looking at outliers	One account drives the number	Report MAE and the largest errors too
Using probabilities from an uncalibrated model	"94% chance" for a lead won one time in four	Check a reliability curve; calibrate if needed
Rebalancing classes then trusting the probabilities	Average prediction of 35% on a 7% problem	Set the threshold by cost instead; if you rebalance, recalibrate
Choosing a threshold on the test set	Test profit is optimistic	Choose on validation or CV; test once
Reading SHAP as cause	"Late payments cause churn"	SHAP explains the model; causes need Chapter 31
Impurity importance as the explanation	Continuous features look most important	Use SHAP or permutation importance
No fairness check	Surprise when a group complains	Recall, precision, and flag rate by every relevant group
Demanding all fairness measures at once	"The model must be equal on everything"	With different base rates that's impossible; choose and document
No model card	The model gets used for something else	Write the card before deployment
Treating validation numbers as forever	Flag rate drifts as the lead mix changes	Monitor calibration and flag rate; retrain on a schedule

In the real world: the rep who scored lower

In June 2026, two months after the lead model goes live, Farah Khan asks for a meeting. Her leads, she says, are "always scored lower than Rahul's", and since the call list is now ordered by score, she's working fewer good leads and her numbers are down.

Meera checks. Farah's leads do score lower on average: 7.1% against Rahul's 9.4%. The model doesn't use the rep's name, so the difference has to come through the features. It does: Farah was given most of the marketplace leads this year, and marketplace leads convert at 1.5%. The model is calibrated within each rep's leads (Farah's leads scored 7–10% are won about 8% of the time). It isn't scoring *her*; it's scoring what she was handed.

That's the honest finding, and it's not the end of the meeting. Two things follow. First, the lead **assignment** was the unfair part, not the model, and the model made a hidden unfairness visible: the score gap is the first hard evidence of it. Vikram rebalances the marketplace leads across reps the same week. Second, Meera adds the check to the model card and the weekly monitoring: mean score and recall by rep, so the next such gap is seen in a dashboard before anyone has to raise it in a meeting.

Farah's numbers recover over the next quarter. She also becomes the model's most useful critic, because she's the one who reads the top-N table every Monday and says when the list feels wrong.

The lesson isn't that the model was right. It's that the evaluation work in this chapter, calibration within groups and recall by group, is what made a fair conversation possible.

Tools

- **scikit-learn** (tested on 1.8.0; current release 1.9.1): `confusion_matrix`, `precision_score`, `recall_score`, `f1_score`, `roc_curve`, `roc_auc_score`, `precision_recall_curve`, `average_precision_score`, `mean_absolute_error`, `root_mean_squared_error`, `mean_absolute_percentage_error`, `r2_score`, `calibration_curve`, `brier_score_loss`, `CalibratedClassifierCV`, `partial_dependence`, and `permutation_importance`.
- **shap** 0.52.0 (`pip install shap`): `LinearExplainer`, `TreeExplainer`, `KernelExplainer`, and its plotting functions (`summary_plot`, `waterfall`), which are the usual way to show SHAP values in a report.
- **imbalanced-learn** 0.14.2 (`pip install imbalanced-learn`): `RandomUnderSampler`, `RandomOverSampler`, `SMOTE`, and a `Pipeline` that resamples only during `fit`.
- **fairlearn** (not used here) provides group-metric tables and mitigation methods if fairness checks become routine.
- Everything ran on one CPU core, Python 3.12.3, on 18 September 2026.
- **Companion files:** `companion/ch39/lead_data.py` (a copy of Chapter 37's helper) rebuilds the lead table. Run the chapter's code from `companion/ch39/`; it also reads `../accounts/accounts.csv`.

The project: cost-based evaluation and explanation of the lead-scoring model

Goal: an evaluation report for a model that a sales head could act on: how many leads to work, what it's worth, where it's uncertain, whom it might treat unfairly, and a model card.

Option A: your own data. Any classifier you built with Chapter 36's workflow, plus two cost numbers you can defend.

Option B: Riverstone. The lead model and validation set.

Steps:

1. **Build the confusion matrix** at 0.5 and at two other thresholds, and compute precision, recall, F1, and specificity by hand for one of them.
2. **Draw ROC and PR curves** and report both areas with the base rate.
3. **Make the top-N table** for N = 25, 50, 100, 200, 400.
4. **Check calibration** with a reliability curve and the Brier score. If you have a second model, compare.
5. **Set costs:** write down the cost of a false positive and the value of a true positive, with your reasoning, as named variables.
6. **Find the threshold** by profit curve and by break-even, and state the range where profit is within 5% of the best.
7. **Add a capacity constraint** and compare the model with the current rule of thumb, in money.
8. **Explain** the model globally (mean |SHAP|) and for three individual leads: the highest-scoring, one at the threshold, and one the model got wrong.
9. **Check fairness** by at least two groupings: flag rate, recall, and precision per group, with a paragraph on what you'd recommend.
10. **Write the model card.**

Stretch goals:

- Test the chosen threshold **on the test set once** and compare the profit with validation.
- Build a **cost-sensitive** version by passing `sample_weight` during training and compare with threshold tuning.
- Use `permutation_importance` on validation data and compare its ranking with SHAP's.
- Draw **SHAP dependence plots** for the two most important features and look for interactions.

You've got it when...

- ☐ I can fill in a confusion matrix from a threshold and compute precision, recall, F1, and specificity by hand.
- ☐ I never report accuracy alone on an imbalanced problem, and I always state the base rate.
- ☐ I can explain why ROC-AUC flatters and PR-AUC doesn't on imbalanced data, and I make the top-N table.
- ☐ I know what MAE, RMSE, MAPE, and R^2 each hide, and I match the metric to what errors cost.
- ☐ I check calibration with a reliability curve before anyone uses a probability as a number, and I can recalibrate a model.

- ☐ I set thresholds from break-even costs, profit curves, or capacity, never from 0.5.
 - ☐ I know that class weights and resampling change probabilities more than rankings, and I recalibrate if I use them.
 - ☐ I can explain a model's top features with SHAP and a single prediction as base value plus contributions.
 - ☐ I can read a partial dependence plot and use it to sanity-check a black-box model.
 - ☐ I run fairness checks by group, and I can explain why calibration, equal opportunity, and predictive parity can't all hold when base rates differ.
 - ☐ I write a model card before a model is used.
-

Recap

- The **confusion matrix** (TP, FP, FN, TN) is the source of **precision**, **recall**, **F1**, and **specificity**. **Accuracy** is misleading whenever classes are imbalanced.
 - **ROC-AUC** measures ranking and ignores the base rate; **PR-AUC** (average precision) reflects what users experience. The **top-N table** is the most useful evaluation for a list.
 - **MAE** is the plain-language error; **RMSE** punishes outliers; **MAPE** breaks near zero and is biased; **R²** depends on the scale. Match the metric to the cost of errors.
 - **Calibration** asks whether predicted probabilities are honest. Check with a **reliability curve** and the **Brier score**; fix with `CalibratedClassifierCV` (Platt or isotonic). Calibration fixes numbers, not rankings.
 - Choose the **threshold** from **break-even** (cost ÷ value), a **profit curve**, or **capacity**. Quote the range, and re-test.
 - **Imbalance** tools (class weights, over- and undersampling, **SMOTE**) mostly inflate probabilities; on a cost-chosen threshold they add little. Fix the threshold first.
 - **SHAP** explains predictions as base value plus feature contributions; **partial dependence** shows a feature's average effect. Both explain the model, not the world.
 - **Fairness**: check flag rate, recall, and precision by group; with different base rates, the fairness definitions conflict, and choosing is a policy decision.
 - A **model card** records purpose, data, performance, limitations, and what the model must not be used for.
-

Practice exercises

Code exercises run from `companion/ch39/` after the chapter's code (they use `p_valid`, `y_valid`, `valid`, `model`, `profit_at`, `WIN_VALUE`, `WORK_COST`, `probs`, `shap_values`, `feature_names`, and the rest). Predict each answer before running it.

Warm-up

1. (hand) A fraud model on 10,000 transactions flags 300; 120 of the flagged are fraud, and 80 frauds were missed. Fill in the confusion matrix and compute precision, recall, and the false positive rate.
2. (hand) Model A has precision 0.9 and recall 0.1; model B has 0.4 and 0.4. Compute both F1 scores. Which would you prefer for (a) a cancer screening test and (b) blocking a credit card?
3. (hand) A win is worth ₹12,000 and following up costs ₹900. What's the break-even probability? Should a lead scored 6% be worked?
4. A forecast has MAE ₹10,000 and RMSE ₹50,000 on the same test set. What does the gap tell you, and what would you look at next?

Core

5. Compute the confusion matrix, precision, and recall at thresholds 0.05, 0.077, and 0.15, and explain the pattern in one sentence.
6. Make the top-N table for the **gradient boosting** probabilities in `probs` and compare it with logistic regression's. Which model would the sales team prefer, and why is the answer not obvious?
7. Change `WORK_COST` to ₹3,000 (a more expensive follow-up process) and recompute the break-even threshold and the profit-maximizing threshold. How much does the best profit fall?
8. Compute the Brier score for a model that predicts the base rate (0.066) for every lead, and for one that predicts 0.5 for every lead. Compare with the three models in section 39.4.
9. Find the highest-scoring **lost** lead in the validation set and explain its score with SHAP. What pushed it up, and would you call the model wrong?
10. Compute recall and precision by `source` at the 0.05 threshold. Which source has the lowest recall, and does the base-rate argument from section 39.8 explain it?

Stretch

- l1. Fit `CalibratedClassifierCV` with `method="sigmoid"` on Naive Bayes and compare its log loss and Brier score with the isotonic version. Which is better here, and why might sigmoid be safer with less data?
- l2. Use `permutation_importance` on the validation set (scoring `"neg_log_loss"`, 10 repeats) and compare its top five features with SHAP's. Where do they disagree, and why can they?
- l3. Apply the 0.077 threshold to the **test** set (train on training plus validation, as Chapter 36 did) and compute profit, worked leads, and wins. How does it compare with validation?

Think about it

- l4. A colleague says: "Our churn model is 91% accurate and the churn rate is 9%." What's your first question, and what two numbers do you ask for instead?
- l5. Marketing wants to show customers "you have a 70% chance of being approved". Which sections of this chapter must you complete before that's responsible, and what would you check monthly?
- l6. The inside sales desk's leads have a recall of 57% against 91% for reps' leads. Propose one policy that addresses it, and say what it costs.

Key terms

confusion matrix · true positive · false positive · false negative · true negative · threshold · accuracy · precision · recall (sensitivity, true positive rate) · specificity · false positive rate · F1 score · harmonic mean · ROC curve · ROC-AUC · precision–recall curve · PR-AUC (average precision) · base rate · top-N table · MAE · RMSE · MAPE · WAPE · R^2 · calibration · reliability curve · Brier score · Platt scaling · isotonic regression · `CalibratedClassifierCV` · break-even threshold · profit curve · capacity constraint · expected value · class imbalance · class weights · undersampling · oversampling · SMOTE · SHAP · Shapley value · base value · local explanation · global importance · permutation importance · partial dependence · ICE plot · fairness · demographic parity · equal opportunity · predictive parity · calibration within groups · feedback loop · model card · monitoring · drift

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 40, Time Series & Forecasting**, measures forecast accuracy with MAPE and WAPE, and backtests over time instead of a single split.
 - **Chapter 42, Recommender Systems & Ranking**, uses precision@k and NDCG, the ranking cousins of the top-N table.
 - **Chapter 44, Capstone**, ends with a model card and a presentation to non-technical leaders.
 - **Chapter 52, Deploying and Monitoring Models**, turns the monitoring row of the model card into dashboards and retraining rules.
 - **Chapter 30, Experiments**, is how threshold changes, fairness fixes, and win-back campaigns are proved to work.
 - **Chapter 31, Causal Inference**, is the answer when someone reads a SHAP plot as a cause.
 - **Chapter 61, Ethics and Responsible Data Practice**, covers fairness and model documentation in depth.
 - **Interview preparation**: the Machine Learning Question Bank (Chapter 74) covers precision versus recall, ROC versus PR, calibration, threshold selection, SMOTE, SHAP, and "how would you explain this model to a manager?", which is asked in nearly every data science interview.
-

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. Flagged 300, of which 120 fraud: TP = 120, FP = 180. Missed frauds: FN = 80. Total fraud = 200, so TN = 10,000 – 300 – 80 = **9,620**. Precision = $120 \div 300 = \mathbf{0.40}$; recall = $120 \div 200 = \mathbf{0.60}$; false positive rate = $180 \div (180 + 9,620) = \mathbf{0.018}$. The base rate is 2%, so a random flag would have 2% precision; 40% is twenty times better, and the team still investigates 180 innocent transactions for every 120 frauds.

2. $F1(A) = 2 \times 0.9 \times 0.1 \div 1.0 = \mathbf{0.18}$; $F1(B) = 2 \times 0.4 \times 0.4 \div 0.8 = \mathbf{0.40}$. (a) Screening wants **recall**: missing a cancer is far worse than a follow-up test, so B, and ideally a model with even higher recall. (b) Blocking a card wants **precision**, because every false block angers a customer and costs support time, so A's precision is attractive, but its 10% recall means it stops almost no fraud. Neither is good; the exercise shows why F1 alone can't choose, and why costs (section 39.5) are the better guide.

3. Break-even = $900 \div 12,000 = \mathbf{0.075}$. A lead at 6% has expected value $0.06 \times 12,000 - 900 = 720 - 900 = -\mathbf{₹180}$, so on average it loses money and shouldn't be worked, unless there's spare capacity that would otherwise be idle, in which case the cost is really lower than ₹900.

4. RMSE five times MAE means a few very large misses dominate; if errors were similar in size the two would be close (for normally distributed errors, RMSE is about $1.25 \times \text{MAE}$). Look at the ten largest errors: are they data problems (a wrong unit, a duplicated order), a segment the model doesn't understand, or unpredictable events? Then decide whether the business cares about those tails, which decides whether RMSE or MAE is the number to optimize.

5.

```
for threshold in [0.05, 0.077, 0.15]:
    tp, fp, fn, tn = matrix_at(threshold)
    print(
        f"threshold {threshold:<5} TP {tp:>3} FP {fp:>4} FN {fn:>3}  "
        f"precision {tp / (tp + fp):.3f}    recall {tp / (tp + fn):.3f}"
    )
```

threshold 0.05	TP 121	FP 783	FN 25	precision 0.134	recall 0.829
threshold 0.077	TP 112	FP 515	FN 34	precision 0.179	recall 0.767
threshold 0.15	TP 76	FP 215	FN 70	precision 0.261	recall 0.521

Raising the threshold trades recall for precision: fewer leads flagged, a higher share of them won, but more wins missed. Between 0.05 and 0.077 the model gives up 9 wins to save 277 follow-ups, which section 39.5 showed is worth it; between 0.077 and 0.15 it gives up another 36 wins to save 336 follow-ups, which isn't.

6.

```
p_boost = probs["gradient boosting"]
order_b = np.argsort(-p_boost)
print("      N    logistic wins    boosting wins")
for k in [50, 100, 200, 400]:
    print(
        f"{k:>6}    {y_valid[np.argsort(-p_valid)[:k]].sum():>13}  "
        f"    {y_valid[order_b[:k]].sum():>13}"
    )
```

N	logistic wins	boosting wins
50	22	23
100	38	33
200	59	50
400	92	80

The lists tie at the top 50 and logistic regression pulls ahead below that: 92 wins against 80 in the top 400, 12 wins out of 146. That's a real-looking gap, and still within what a different six months could reverse. The sales team would pick the list that looks better this quarter, which is a weak basis: choose on calibration, speed, and explainability (Chapter 37's reasons), and if it matters, run both for a period and compare (Chapter 30).

7.

```
WORK_COST = 3000
print(f"break-even probability: {WORK_COST / WIN_VALUE:.4f}")
profits_3k = np.array([profit_at(t)[0] for t in grid])
print(
    f"best threshold {grid[profits_3k.argmax():.3f]    profit ₹{profits_3k.max():.0f}    "
    f"(was ₹{profits.max():.0f} at ₹1,500 per lead)"
)
WORK_COST = 1500
```

```
break-even probability: 0.0992
best threshold 0.103    profit ₹1,633,500    (was ₹2,448,060 at ₹1,500 per lead)
```

Doubling the cost of working a lead doubles the break-even probability (to about 0.099) and moves the profit-maximizing threshold up to match, and the best profit falls by roughly ₹0.9 million. The threshold is a function of the costs, which is the point: when the process changes, re-run this, not the model.

8.

```
for name, p in [
    ("base rate for everyone", np.full(len(y_valid), y_valid.mean())),
    ("0.5 for everyone", np.full(len(y_valid), 0.5)),
]:
    print(f"{name:<24} Brier {brier_score_loss(y_valid, p):.4f}")
```

```
base rate for everyone    Brier 0.0613
0.5 for everyone          Brier 0.2500
```

Predicting the base rate scores 0.0613, and the three real models score 0.053 (logistic and boosting) to 0.124 (Naive Bayes). So logistic regression beats the know-nothing benchmark by a modest margin, and Naive Bayes is **worse than predicting the base rate**: a constant 6.6% would be a better probability than its confident numbers. Predicting 0.5 for everyone scores 0.25, the worst possible constant.

9.

```
lost = np.where(y_valid == 0)[0]
worst = lost[np.argmax(p_valid[lost])]
lead = valid.iloc[worst]
print(
    f"lead {lead['lead_id']}: {lead['source']}, "
    f"{lead['segment']}, size {lead['company_size']}, "
    f"predicted {p_valid[worst]:.1%}, lost"
)
pushes = pd.Series(shap_values[worst], index=feature_names)
print(pushes[pushes.abs() > 0.2].sort_values(ascending=False).round(3).to_string())
```

```

lead 107979: Trade fair, Wholesale, size 51-200, predicted 56.1%, lost
num__text_strong          0.693
cat__source_Marketplace   0.467
num__log_quantity         0.411
num__website_visits       0.329
num__free_email           0.237
cat__source_Trade fair    0.235
cat__segment_Retail       0.223
num__activities_24h       0.201
cat__product_interest_Industrial -0.213

```

The lead has every sign of a good one, which is why it scored high: the same features that mark most won leads. It was still lost. That isn't a model error in any useful sense: a well-calibrated 50–60% prediction means the lead is lost about half the time, and this is one of those. "Wrong" would be a pattern: a group of high-scoring leads that lose far more often than their scores say, which is what the reliability curve checks.

10.

```

by_source = valid_score.groupby("source").apply(
    lambda g: pd.Series(
        {
            "leads": len(g),
            "win rate": g["won"].mean(),
            "share called": g["called"].mean(),
            "recall": g.loc[g["won"] == 1, "called"].mean(),
            "precision": (
                g.loc[g["called"], "won"].mean() if g["called"].any() else np.nan
            ),
        }
    )
)
print(by_source.sort_values("recall").round(3).to_string())

```

source	leads	win rate	share called	recall	precision
Marketplace	1000.0	0.014	0.155	0.214	0.019
Website	554.0	0.065	0.401	0.722	0.117
Cold call	140.0	0.086	0.586	0.750	0.110
Trade fair	233.0	0.133	0.837	0.968	0.154
Referral	194.0	0.211	0.928	1.000	0.228
Partner	104.0	0.115	0.673	1.000	0.171

Marketplace leads have the lowest recall by far, and the lowest base rate: a calibrated model gives most of them scores below 5%, so the winnable ones are missed along with the rest. The base-rate argument explains the number, and the equal-opportunity view still objects to it, exactly as with the inside sales desk (which handles most marketplace leads). The two findings are the same finding seen through two groupings.

11.

```

sigmoid_nb = CalibratedClassifierCV(nb, method="sigmoid", cv=5).fit(
    train[X_cols], train["won"])
)
p_sig = sigmoid_nb.predict_proba(valid[X_cols])[:, 1]
print(
    f"sigmoid: log loss {log_loss(y_valid, p_sig):.4f} "
    f" Brier {brier_score_loss(y_valid, p_sig):.4f}"
)
print(
    f"isotonic: log loss {log_loss(y_valid, p_nb_cal):.4f} "
    f" Brier {brier_score_loss(y_valid, p_nb_cal):.4f}"
)

```

```

sigmoid: log loss 0.2063    Brier 0.0552
isotonic: log loss 0.2022    Brier 0.0544

```

Isotonic does slightly better here, because Naive Bayes's distortion isn't a simple S-shape and isotonic can bend to fit it. Sigmoid fits only three parameters, so it can't overfit the calibration data; with a few hundred positives or fewer, isotonic's step function can chase noise and make calibration *worse* on new data. Rule of thumb: sigmoid below about a thousand positives, isotonic above.

12.

```

from sklearn.inspection import permutation_importance

perm = permutation_importance(
    model, valid[X_cols], y_valid, scoring="neg_log_loss", n_repeats=10, random_state=39
)
perm_series = pd.Series(perm.importances_mean, index=X_cols).sort_values(
    ascending=False
)
print("permutation importance (increase in log loss when shuffled), top 5:")
print(perm_series.head(5).round(4).to_string())

```

```

permutation importance (increase in log loss when shuffled), top 5:
source                0.0334
company_size          0.0092
log_quantity          0.0073
segment              0.0034
activities_24h        0.0029

```

Permutation importance works on the **raw columns**, so `source` appears once, while SHAP scores each one-hot column (marketplace, website, referral) separately; that's the main reason the lists look different. They can also disagree in substance: permutation importance measures how much the score degrades without a feature, so a feature that's highly correlated with another can look unimportant (the other covers for it), while SHAP still assigns it a share. Neither is wrong; they answer different questions.

13.

```

final_model = make_model(LEAD_CATS, LEAD_NUMS).fit(
    pd.concat([train, valid])[X_cols], pd.concat([train, valid])["won"]
)
p_test = final_model.predict_proba(test[X_cols])[:, 1]
y_test = test["won"].to_numpy()
profit_t, worked_t, wins_t = profit_at(0.077, p_test, y_test)
print(
    f"test set ({len(test):,} leads, {y_test.sum():,} "
    f"won): {worked_t} worked, {wins_t} wins, "
    f"profit ₹{profit_t:,.0f}"
)
print(
    f"per lead: ₹{profit_t / len(test):,.0f} on test vs "
    f"₹{profits.max() / len(valid):,.0f} on validation"
)

```

```

test set (1,185 leads, 104 won): 332 worked, 80 wins, profit ₹1,922,400
per lead: ₹1,622 on test vs ₹1,100 on validation

```

Per lead, the test profit is higher than validation's, because the test period's win rate is higher (8.8% against 6.6%) and the model, retrained on more data, ranks it well. The reassurance is that it didn't fall: the threshold chosen on validation transferred. A large drop here would have meant the validation threshold was tuned to noise.

14. First question: "What does predicting 'no churn' for everyone score?" At a 9% churn rate, that's 91% accurate, the same as the model, so the headline says nothing. Ask for **recall at the threshold you'd act on** (what share of churners does it catch?) and **precision** (of the accounts flagged, how many churn?), or equivalently a top-N table: of the 100 highest-risk accounts, how many churned?

15. Sections 39.4 and 39.8, at minimum: the probability must be **calibrated** (a reliability curve, and a Brier score against the base rate), and it must be calibrated **within each group** the customer might belong to, or the number is a lie to some of them. Section 39.9's model card should say what the number means and what it's not for. Monthly: the reliability curve on recently decided applications, the mean predicted against the actual approval rate overall and by group, and the share of customers shown each band of probability, since a drift in the applicant mix will move all of these before anyone notices.

16. One option: **a separate threshold for inside-desk leads**, set so their recall matches the reps' (around 91%). Cost: the desk would work many more leads with lower precision, so the follow-up cost per win rises; at a 3.7% base rate that may mean working most desk leads, which is what happened before the model. A cheaper alternative is a **floor**: the desk always works its top 15% of leads by score, whatever the threshold says, which bounds the cost. The honest answer to "what does it cost" is a number from `profit_at` under each policy, which is exercise 7's method applied to one group.